Capstone Deployment Architecture, Design Rationale, and Pre-Deployment Checklist

1. Deployment Architecture Diagram

Below is the deployment architecture diagram used for the forecasting application:

Key Architectural Principles:

The server performs no heavy computation; all Monte Carlo modeling is done offline.Flask only evaluates formula-based models, guaranteeing low latency. Static assets (charts, maps, tables) act as the presentation layer for precomputed results. System remains affordable, lightweight, and scalable.

2. System Design Rationale

Overview

This system supports an EV adoption forecasting platform combining Monte Carlo simulation, time-series modeling, and non-linear curve fitting. All heavy computation is performed offline, while the deployed application evaluates mathematical formulas and displays pre-generated charts and maps.

Design Decisions

1. Web Framework (Flask)

Flask was chosen for its minimal footprint, simple routing, and suitability for academic modeling tools. Since the application only evaluates lightweight mathematical expressions, a larger framework such as Django or FastAPI was unnecessary.

2. Removal of Pandas for Deployment

Pandas was removed to reduce RAM usage, eliminate long Render build times, and improve app cold-start speed. Excel parsing is handled entirely through OpenPyXL, which is lighter and more deployment-friendly.

3. Static Pre-Computed Assets

All Monte Carlo outputs, county EV curves, and spatial maps are generated offline. The server only loads:

- Cleaned formula tables
- County-level charts
- GIS population–supercharger heatmaps

This ensures extremely low compute cost during runtime and keeps the application stateless.

4. Safe Formula Evaluation Engine

A restricted evaluator is used to translate the fitted mathematical curves.Only functions such as x, exp, log, and sqrt are permitted. This prevents arbitrary code execution while supporting all cubic, logistic, and exponential forecasting equations.

5. Render Hosting

Render provides:

- Free-tier hosting suitable for coursework and prototyping
- Automatic HTTPS
- GitHub-linked CI/CD
- Fast container-based deployment

The application's CPU, RAM, and disk requirements fall well below free-tier limits.

Cost Estimate

- Development: 30-40 hours (modeling + Flask + deployment)
- Compute: Performed locally → $0 cloud compute cost
- Hosting: Render Free Web Service → $0/month
- Optional S3 storage upgrade → $1–$2/month

Total recurring cost: $0 at current scale. This architecture balances academic explainability, low operating cost, and clean separation between offline modeling and online forecasting.

3. Model Lifecycle: 5-Year Update & Recalibration Cycle

The EV forecasting model operates on a fixed 5-year cycle, preserving the long-term 2024–2050 blueprint while adapting to real-world changes.

What "5-Year Cycle" Means

Years 1–4 — Observation Phase
Planners monitor:
• EV adoption vs forecast band
• New supercharger installations
• Population updates
• Climate target progression
If adoption lags, charger build-out can be increased within the existing envelope without full recalculation.

Year 5 — Recalibration Phase
System updates:
• Population baselines
• Charger counts
• EV VIN registrations
• Policy targets
• Behavioral parameters
Outputs new Lower/Upper curves + Monte Carlo forecast (P10/P50/P90).

Why Recalibrate Every 5 Years?
• Strategic long horizon
• Short-term noise absorbed by boundaries
• Infrastructure budgets follow multi-year cycles
• Multi-year data provides better signal

4. Pre-Deployment Checklist

A. Application Structure
- app.py runs locally without errors
- templates/index.html present
- static/data, static/charts, static/maps directories included
- Excel input files + PNG outputs committed to repo

B. Dependencies
- requirements.txt includes Flask, gunicorn, openpyxl
- pandas/numpy removed to avoid heavy dependencies
- Secret key defined

C. Render Configuration
- Correct build command: pip install -r requirements.txt
- Correct start command: gunicorn app:app
- Python version selected automatically
- Root directory mapped correctly

D. Functional Testing

- Homepage loads
- Valid county returns results
- Formula evaluation returns numeric outputs
- Chart and map images load correctly
- Flash messages show for invalid inputs

E. Safety & Reliability

- Restricted eval implementation verified
- No user-uploaded files
- No database credentials
- Static assets load with correct relative paths

F. Final Verification

- Deployed app responds in <200ms
- Mobile formatting validated
- No missing images or Excel files